

Интеграция сенсоров и базовое управление роботом в цифровом двойнике

ROS2 Humble + Gazebo Classic + TurtleBot3

Цель работы: Изучение механизмов межпроцессного взаимодействия в ROS2 через систему топиков. Освоение работы с сенсорными данными лидара (LaserScan), настройка QoS-профилей, реализация реактивного поведения робота на основе данных proximity-детекции, визуализация данных в RViz2.

Ход работы

1.1. Архитектура Publisher-Subscriber в ROS2

ROS2 строится на парадигме слабосвязанного взаимодействия. Процессы (ноды) не вызывают друг друга напрямую — они публикуют данные в именованные каналы (топики) и подписываются на них независимо.

Это даёт три важных свойства. Во-первых, нода не знает, кто читает её данные — один подписчик или десять. Во-вторых, издатель и подписчик могут запускаться и останавливаться в любом порядке. В-третьих, один и тот же топик `/scan` одинаково работает и с реальным лидаром TurtleBot3, и с виртуальным сенсором Gazebo — код ноды не меняется.

1.2. Сообщение LaserScan

Топик `/scan` передаёт сообщения типа `sensor_msgs/msg/LaserScan`. Ключевые поля:

`range_min` и `range_max` — минимальная и максимальная рабочая дальность лидара в метрах. Значения вне этого диапазона считаются невалидными.

`ranges` — массив `float32`, где каждый элемент — расстояние до препятствия в одном угловом направлении. Для лидара LDS-01 (TurtleBot3) массив содержит 360 элементов (один градус на элемент). Значения `inf` означают отсутствие препятствия в пределах дальности.

`angle_min`, `angle_max`, `angle_increment` — угловой диапазон и шаг сканирования в радианах.

При обработке нельзя брать минимум из всего массива без фильтрации — `nan` и `inf` испортят результат. Правильный паттерн:

```
python
```

```
valid = [r for r in msg.ranges if msg.range_min < r < msg.range_max]
```

```
min_dist = min(valid) if valid else float('inf')
```

1.3. Сообщение Twist и топик /cmd_vel

Управление движением робота осуществляется через топик `/cmd_vel` с типом `geometry_msgs/msg/Twist`. Сообщение содержит два вектора:

linear — линейная скорость по осям x, y, z в м/с. Для наземного робота используется только linear.x (движение вперёд/назад).

angular — угловая скорость по осям в рад/с. Для наземного робота используется только angular.z (поворот вокруг вертикальной оси).

Дифференциальный привод TurtleBot3 вычисляет скорости колёс из этих двух значений: левое колесо = $\text{linear.x} - \text{angular.z} * L/2$, правое = $\text{linear.x} + \text{angular.z} * L/2$, где L — расстояние между колёсами.

1.4. QoS-профили в ROS2

Quality of Service — набор политик, определяющих как именно доставляются сообщения. Два ключевых параметра:

Reliability — RELIABLE гарантирует доставку каждого сообщения (с повторными попытками), BEST_EFFORT отправляет без гарантий. Сенсоры Gazebo публикуют данные с профилем BEST_EFFORT, поэтому подписчик обязан использовать тот же профиль — иначе соединение не установится и данные не придут. Это частая причина ошибки когда нода запущена, топик есть, но callback никогда не вызывается.

Durability — VOLATILE означает что новый подписчик не получит старые сообщения, TRANSIENT_LOCAL — получит последнее сохранённое. Для потоковых сенсорных данных всегда VOLATILE.

1.5. Таймер публикации vs реактивная публикация

В brain_node.py используется два независимых механизма. Таймер (create_timer) публикует /cmd_vel с фиксированной частотой 10 Гц независимо от того, приходят ли данные лидара. Это важно — контроллер робота ожидает регулярный поток команд, и их отсутствие интерпретируется как потеря связи.

Callback лидара (_scan_callback) вызывается каждый раз при получении нового скана — обычно 5–10 Гц. Он только обновляет внутреннее состояние (_wall_detected), но не публикует напрямую. Разделение логики восприятия и публикации команд — правильный архитектурный паттерн.

1.6. Добавление виртуального лидара в Gazebo

Чтобы Gazebo генерировал данные /scan, в SDF-описание робота нужно добавить плагин лидара. Минимальная конфигурация для сферического лидара 360°:

xml

```
<sensor name="lidar" type="ray">
  <always_on>true</always_on>
  <update_rate>10</update_rate>
  <ray>
    <scan>
      <horizontal>
        <samples>360</samples>
        <min_angle>-3.14159</min_angle>
        <max_angle>3.14159</max_angle>
      </horizontal>
    </scan>
    <range>
      <min>0.12</min>
      <max>3.5</max>
    </range>
  </ray>
  <plugin name="gazebo_ros_ray_sensor" filename="libgazebo_ros_ray_sensor.so">
    <ros>
      <remapping>~/out:=scan</remapping>
    </ros>
    <output_type>sensor_msgs/LaserScan</output_type>
    <frame_name>base_scan</frame_name>
  </plugin>
</sensor>
```

2.1. Добавление робота в симуляцию

Проще всего использовать готовую модель TurtleBot3 Burger из пакета turtlebot3_gazebo.
Установить:

```
bash
```

```
sudo apt install ros-humble-turtlebot3-gazebo ros-humble-turtlebot3-description
```

```
export TURTLEBOT3_MODEL=burger
```

Добавить в lab_scene.world перед закрывающим тегом </world>:

xml

```
<include>

  <uri>model://turtlebot3_burger</uri>

  <name>turtlebot3</name>
```

```
<pose>0 0 0.01 0 0 0</pose>
```

```
</include>
```

Добавить в GAZEBO_MODEL_PATH путь к моделям turtlebot3:

```
bash
```

```
export
```

```
GAZEBO_MODEL_PATH=$GAZEBO_MODEL_PATH:/opt/ros/humble/share/turtlebot3_gazebo/models
```

2.2. Запуск симуляции с роботом

```
bash
```

```
source ~/ros2_ws/install/setup.bash
```

```
export TURTLEBOT3_MODEL=burger
```

```
ros2 launch lab_digital_twin simulation.launch.py
```

В Gazebo должен появиться робот TurtleBot3 в центре сцены.

2.3. Верификация потока данных лидара

```
bash
```

```
# Проверить что /scan публикуется
```

```
ros2 topic hz /scan
```

```
# Ожидаем: average rate: 10.000
```

```
# Посмотреть структуру сообщения
```

```
ros2 topic echo /scan --once
```

```
# Проверить подключение brain_node к топику
```

```
ros2 topic info /scan
```

```
# Ожидаем: Publisher count: 1, Subscription count: 1
```

2.4. Проверка реакции на стены

Запустить телеуправление в третьем терминале:

```
bash
```

```
sudo apt install ros-humble-turtlebot3-teleop
```

```
ros2 run turtlebot3_teleop teleop_keyboard
```

Подъехать роботом к стене на расстояние менее 0.5 м. В консоли brain_node должно появиться предупреждение о детекции стены.

Проверить что команды скорости меняются при детекции:

```
bash
```

```
ros2 topic echo /cmd_vel
```

При детекции стены оба вектора должны обнуляться.

2.5. Визуализация в RViz2

```
bash
```

```
ros2 run rviz2 rviz2
```

В интерфейсе RViz2:

1. Установить Fixed Frame: odom (или base_link если TF не настроен)
2. Нажать Add - By topic /scan - LaserScan
3. Установить Size: 0.03, Color: по интенсивности или фиксированный

Теперь облако точек лидара отображается поверх сцены в реальном времени.

Приближение к стене должно быть видно как уплотнение точек с одной стороны.

2.6. Расширение brain_node — секторная детекция

Базовая детекция берёт минимум по всему скану. Доработать _scan_callback для секторной детекции — отдельно для передней полусферы (опасность столкновения) и задней (свободно для отступления):

```
python
```

```
def _scan_callback(self, msg: LaserScan) -> None:
```

```
    ranges = list(msg.ranges)
```

```
    n = len(ranges)
```

```
    def sector_min(start_deg, end_deg):
```

```
        indices = range(
```

```
            int(start_deg * n / 360),
```

```
            int(end_deg * n / 360)
```

```
        )
```

```
        vals = [ranges[i] for i in indices]
```

```
if msg.range_min < ranges[i] < msg.range_max]
return min(vals) if vals else float('inf')
```

```
front = sector_min(330, 360) # ±30° перед роботом
# (индексы 330-360 и 0-30 — нужно объединить оба куска)
```

```
self._wall_detected = front < WALL_DISTANCE_M
```

```
if self._wall_detected:
    self.get_logger().warn(
        f"Препятствие спереди: {front:.2f} м"
    )
```